

GamutX

CLI Tool Edge Cases, Flags

PYTHON SCRIPTING

Handling Edge Cases in CLI Tools

- Overview of the Script
- Common Mistakes in CLI Usage
- Edge Case 0: Check if File Exists
- Edge Case 1: First Argument Must Be a Valid File
- Edge Case 2: The Second Argument Must Be a Valid Filter
- Edge Case 3: Ensuring the Correct Number of Arguments
- Edge Case 4: Close Program if Exception Fails

Overview of the Script

Before we dive into **error handling**, let's first understand the **base script** we're working with.

Base Script

```
import sys
```

```
first_arg = sys.argv[1]
with open(first_arg, 'r') as f:
    data = f.readlines()
```

Takes a file path as the **first argument** (e.g., **log.txt**)

```
second_arg = sys.argv[2]
```

Takes a filter keyword as the **second argument** (e.g., **INFO** or **ERROR**)

```
if second_arg == "INFO":
    for each_line in data:
        if "INFO" in each_line.strip():
            print(each_line.strip())
```

```
if second_arg == "ERROR":
    for each_line in data:
        if "ERROR" in each_line.strip():
            print(each_line.strip())
```

Filters and prints all lines that contain the keyword

- This works — **but only if the user gets everything right.**

Common Mistakes in CLI Usage

This script works **only** if the user provides **correct inputs in the right order**.

✓ Expected Input

```
python script.py log.txt ERROR
```

```
python script.py log.txt ERROR
```

```
python script.py log.txt ERROR
```

```
python script.py log.txt INFO
```

User Input

→ `python script.py ERROR log.txt`

→ `python script.py log.txt ERROR INFO`

→ `python script.py log.txt ERRORS`

→ `python script.py log.txt`

Keyword passed before the file path



1

2

3

expected 2 Args, given 3 Args

Unknown Keyword

Args 0

Args 1

Args 2

Second argument (keyword) is missing

✗ Mistake

→ **Wrong order of arguments**

→ **Too many arguments**

→ **Invalid keyword**

→ **Missing arguments**

But In a **real studio** or **pipeline**, These **mistakes** are **very common**
so it's **essential to handle them gracefully**.



Edge Case 0: Check if File Exists

Scenario

- A user wants to filter logs for **ERROR** entries, but they mistakenly provide a **non-existent** file name.

```
python script.py non_existent_file.txt ERROR
```

provide non existed file

● What Goes Wrong

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
⊗ PS C:\Users\pralhad\Desktop\My Script> python script.py non_existent_file.txt ERROR
Traceback (most recent call last):
  File "C:\Users\pralhad\Desktop\My Script\script.py", line 8, in <module>
    with open(first_arg, 'r') as f:
FileNotFoundError: [Errno 2] No such file or directory: 'non_existent_file.txt'
○ PS C:\Users\pralhad\Desktop\My Script> █
```

✗ The program fails because it's **trying to open a file that isn't there**.

 ✓ Fix: Add a File Existence Check

We Use Python's **os.path.isfile()** before opening the file:

```
import os
import sys

first_arg = sys.argv[1]
if not os.path.isfile(first_arg):
    print(f"ERROR: The file '{first_arg}' does not exist. Please provide a valid file path.")
    sys.exit(1)

with open(first_arg, 'r') as f:
    data = f.readlines()

second_arg = sys.argv[2]

if second_arg == "INFO":
    for each_line in data:
        if "INFO" in each_line.strip():
            print(each_line.strip())

if second_arg == "ERROR":
    for each_line in data:
        if "ERROR" in each_line.strip():
            print(each_line.strip())
```

Check if the file exists

Stops execution early, 1 = failure

Testing the Fix

Test Case 1: Non-existent file

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
⊗ PS C:\Users\pralhad\Desktop\My Script> python script.py non_existent_file.txt ERROR
  ERROR: The file 'non existent file.txt' does not exist. Please provide a valid file path.
○ PS C:\Users\pralhad\Desktop\My Script>
```

Providing Non existing file

What was the error

What need to be done

Test Case 2: Valid file

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\pralhad\Desktop\My Script> python script.py log.txt ERROR
2024-07-23 09:15:00 ERROR: FileNotFoundError: Unable to locate the configuration file.
2024-07-23 09:20:00 ERROR: OutOfMemory: System ran out of memory while processing the data.
2024-07-23 09:35:00 ERROR: NullReference: Attempted to access an object that is null.
2024-07-23 10:10:00 ERROR: IOException: Error occurred while reading from the disk.
2024-07-23 10:25:00 ERROR: NetworkTimeout: Connection to the server timed out.
○ PS C:\Users\pralhad\Desktop\My Script>
```

Valid File

Edge Case 1: First Argument Must Be a Valid File

Scenario

- A user accidentally passes the **filter keyword** first and the **filename second**, like this:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Ⓢ PS C:\Users\pralhad\Desktop\My Script> python script.py INFO log.txt
Traceback (most recent call last):
  File "C:\Users\pralhad\Desktop\My Script\script.py", line 8, in <module>
    with open(first_arg, 'r') as f:
FileNotFoundError: [Errno 2] No such file or directory: 'INFO'
Ⓢ PS C:\Users\pralhad\Desktop\My Script> 
```

Let's See How we can Fix this

To handle this, we'll use a **try-except block**.

```
import sys

try:
    first_arg = sys.argv[1]
    with open(first_arg, 'r') as f:
        data = f.readlines()
except FileNotFoundError:
    print("ERROR: A valid file path is expected as the first argument.")
    sys.exit(1)

second_arg = sys.argv[2]

if second_arg == "INFO":
    for each_line in data:
        if "INFO" in each_line.strip():
            print(each_line.strip())

if second_arg == "ERROR":
    for each_line in data:
        if "ERROR" in each_line.strip():
            print(each_line.strip())
```

Tries to read the file safely

Catches the error if the file doesn't exist

Stops execution to avoid additional failure

Testing the Fix

Test Case 1: Invalid file path

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
⊗ PS C:\Users\pralhad\Desktop\My Script> python script.py INFO log.txt
  ERROR: A valid file path is expected as the first argument.
○ PS C:\Users\pralhad\Desktop\My Script>
```

It should be file path not filter keyword

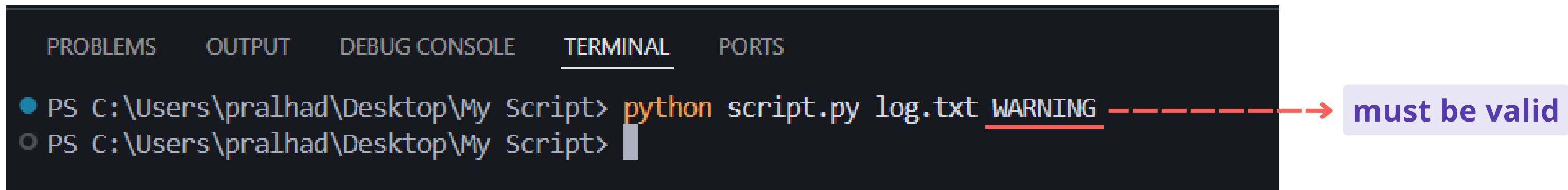
Test Case 2: Valid file path

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\pralhad\Desktop\My Script> python script.py log.txt INFO
  2024-07-23 09:05:00 INFO: Application started successfully.
  2024-07-23 09:25:00 INFO: Memory cleanup process initiated.
  2024-07-23 09:30:00 INFO: Memory cleanup completed successfully.
  2024-07-23 09:45:00 INFO: Attempting to restart the application.
  2024-07-23 09:55:00 INFO: Application restarted successfully.
  2024-07-23 10:15:00 INFO: Disk read operation retried successfully.
  2024-07-23 10:30:00 INFO: Network connection re-established.
  2024-07-23 10:40:00 INFO: Diagnostics completed with no significant issues found.
  2024-07-23 10:50:00 INFO: Security protocols engaged and access blocked.
○ PS C:\Users\pralhad\Desktop\My Script>
```

Edge Case 2: The Second Argument Must Be a Valid Filter

Scenario

Now, imagine a user wants to filter log entries, but instead of passing **INFO** or **ERROR**, they pass something invalid — like **DEBUG** or **WARNING**.



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\pralhad\Desktop\My Script> python script.py log.txt WARNING
○ PS C:\Users\pralhad\Desktop\My Script>
```

● What Goes Wrong

- The program doesn't crash, but prints nothing
- No error, no result

This is confusing. Users don't know **if the script ran successfully** or **if something went wrong**.

✓ Fix: Validate the Filter Keyword

- We'll fix this by **validating** the **second argument**.
- If it's not one of the accepted filters (INFO or ERROR), the program will stop and display a clear error message.

```
import sys

try:
    first_arg = sys.argv[1]
    with open(first_arg, 'r') as f:
        data = f.readlines()
except FileNotFoundError:
    print("ERROR: A valid file path is expected as the first argument.")
    sys.exit(1)

second_arg = sys.argv[2]
accepted_args = ["INFO", "ERROR"]

if second_arg in accepted_args:
    if second_arg == "INFO":
        for each_line in data:
            if "INFO" in each_line.strip():
                print(each_line.strip())
    elif second_arg == "ERROR":
        for each_line in data:
            if "ERROR" in each_line.strip():
                print(each_line.strip())
else:
    raise ValueError(f"Invalid argument: {second_arg}. Accepted values are 'INFO' or 'ERROR'.")
```

Restricts filter to known values only

stops execution with a clear message

Testing the Fix

Test Case 1: Valid filter

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\pralhad\Desktop\My Script> python script.py log.txt INFO
2024-07-23 09:05:00 INFO: Application started successfully.
2024-07-23 09:25:00 INFO: Memory cleanup process initiated.
2024-07-23 09:30:00 INFO: Memory cleanup completed successfully.
2024-07-23 09:45:00 INFO: Attempting to restart the application.
2024-07-23 09:55:00 INFO: Application restarted successfully.
2024-07-23 10:15:00 INFO: Disk read operation retried successfully.
```

valid filter

Test Case 2: Invalid filter

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
⊗ PS C:\Users\pralhad\Desktop\My Script> python script.py log.txt WARNING
Traceback (most recent call last):
  File "C:\Users\pralhad\Desktop\My Script\script.py", line 24, in <module>
    raise ValueError(f"Invalid argument: {second_arg}. Accepted values are 'INFO' or 'ERROR'.")
ValueError: Invalid argument: WARNING. Accepted values are 'INFO' or 'ERROR'.
○ PS C:\Users\pralhad\Desktop\My Script> 
```

invalid filter

Raise Value Error

Edge Case 3: Ensuring the Correct Number of Arguments

Scenario

Our CLI tool expects exactly two arguments:

- A log file path
- A filter keyword (INFO or ERROR)

But what happens if the **user forgets one or adds an extra Argument** ?

What if User Forget ?

```
python script.py log.txt ____
```

----> Forget 1 Argument

```
python script.py log.txt ERROR INFO
```

----> Adds Extra Arguments

Let's test these cases:

✗ Too Few Arguments

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
⊗ PS C:\Users\pralhad\Desktop\My Script> python script.py log.txt
Traceback (most recent call last):
  File "C:\Users\pralhad\Desktop\My Script\script.py", line 11, in <module>
    second_arg = sys.argv[2]
IndexError: list index out of range
○ PS C:\Users\pralhad\Desktop\My Script>
```

Filter Keyword missing

Why?

- The script tries to access `sys.argv[2]`, but it wasn't provided.

Too Many Arguments

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\pralhad\Desktop\My Script> python script.py log.txt INFO EXTRA_ARG
2024-07-23 09:05:00 INFO: Application started successfully.
2024-07-23 09:25:00 INFO: Memory cleanup process initiated.
2024-07-23 09:30:00 INFO: Memory cleanup completed successfully.
2024-07-23 09:45:00 INFO: Attempting to restart the application.
2024-07-23 09:55:00 INFO: Application restarted successfully.
2024-07-23 10:15:00 INFO: Disk read operation retried successfully.
2024-07-23 10:30:00 INFO: Network connection re-established.
2024-07-23 10:40:00 INFO: Diagnostics completed with no significant issues found.
2024-07-23 10:50:00 INFO: Security protocols engaged and access blocked.
○ PS C:\Users\pralhad\Desktop\My Script>
```

- No error, but the script accepts more than expected,
- which could cause **unexpected behavior** later.

✓ Fix: Check Number of Arguments

```
import sys

try:
    if len(sys.argv) < 3:
        raise ValueError("ERROR: At least two arguments are required: a file path and a filter (INFO/ERROR).")
    if len(sys.argv) > 3:
        raise ValueError("ERROR: Too many arguments. Only two arguments are allowed.")

    first_arg = sys.argv[1]

    with open(first_arg, 'r') as f:
        data = f.readlines()
except FileNotFoundError:
    print("ERROR: A valid file path is expected as the first argument.")
    sys.exit(1)

second_arg = sys.argv[2]
accepted_args = ["INFO", "ERROR"]

if second_arg in accepted_args:
    if second_arg == "INFO":
        for each_line in data:
            if "INFO" in each_line.strip():
                print(each_line.strip())
    elif second_arg == "ERROR":
        for each_line in data:
            if "ERROR" in each_line.strip():
                print(each_line.strip())
else:
    raise ValueError(f"Invalid argument: {second_arg}. Accepted values are 'INFO' or 'ERROR'.")
```

It Ensures the right number of arguments is passed

Helps users understand

- *what they did wrong*
- *How fix it*

How does it work?

- If `len(sys.argv) < 3`, the program raises:

```
ValueError: ERROR: At least two arguments are required: a file path and a filter (INFO/ERROR).
```

- If `len(sys.argv) > 3`, the program raises:

```
ValueError: ERROR: Too many arguments. Only two arguments are allowed.
```

Testing the Fix

Test Case 1: Too Few Arguments

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\pralhad\Desktop\My Script> python script.py log.txt _____ --> Missing Args
Traceback (most recent call last):
  File "C:\Users\pralhad\Desktop\My Script\script.py", line 6, in <module>
    raise ValueError("ERROR: At least two arguments are required: a file path and a filter (INFO/ERROR).")
ValueError: ERROR: At least two arguments are required: a file path and a filter (INFO/ERROR).
PS C:\Users\pralhad\Desktop\My Script> |
```

Test Case 2: Too Many Arguments

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\pralhad\Desktop\My Script> python script.py log.txt INFO EXTRA_ARG --> Extra Args
Traceback (most recent call last):
  File "C:\Users\pralhad\Desktop\My Script\script.py", line 8, in <module>
    raise ValueError("ERROR: Too many arguments. Only two arguments are allowed.")
ValueError: ERROR: Too many arguments. Only two arguments are allowed.
PS C:\Users\pralhad\Desktop\My Script> |
```

Why use this method?

- It ensures the **program receives** exactly the input it **expects**.
- It prevents **unexpected errors** or behavior caused by **too few** or **too many arguments**.
- It gives **clear guidance** to the **user** about what went wrong and how to **fix** it.

Edge Case 4: Close Program if Exception Fails

Scenario

Sometimes, your script might run into unexpected problems, like:

- A **bug** in the code
- A user passing **incorrect input**
- A runtime error (e.g. **typos**, **syntax mistake**)

Without proper handling, Python will show a **raw traceback**, which is confusing for **non-technical users**.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\pralhad\Desktop\My Script> python script.py log.txt INFO EXTRA_ARG
Traceback (most recent call last):
  File "C:\Users\pralhad\Desktop\My Script\script.py", line 8, in <module>
    raise ValueError("ERROR: Too many arguments. Only two arguments are allowed.")
ValueError: ERROR: Too many arguments. Only two arguments are allowed.
PS C:\Users\pralhad\Desktop\My Script> 
```

The Goal

We want to:

- Stop the program **gracefully**
 - Show the user a **clear error message**
 - Avoid raw and messy tracebacks
 - Exit with a proper **exit code** (**1 means "error"**)
- 

The Goal

We want to:

- Stop the program **gracefully**
- Show the user a **clear error message**
- Avoid raw and messy tracebacks
- Exit with a proper **exit code** (**1 means "error"**)



Approach

To handle these scenarios,
we'll use **try-except** blocks along with **sys.exit()**.

- Wrap the main logic inside a try block.
- Catch known errors (**ValueError**, **FileNotFoundError**) and provide friendly messages.
- Add a generic **except** to catch anything else.

```

import sys
import os

try:
    if len(sys.argv) < 3:
        raise ValueError("ERROR: At least two arguments are required: a file path and a filter (INFO/ERROR).")
    if len(sys.argv) > 3:
        raise ValueError("ERROR: Too many arguments. Only two arguments are allowed.")

    first_arg = sys.argv[1]
    if not os.path.isfile(first_arg):
        raise FileNotFoundError(f"ERROR: The file '{first_arg}' does not exist. Please provide a valid file path.")

    with open(first_arg, 'r') as f:
        data = f.readlines()

    second_arg = sys.argv[2]
    accepted_args = ["INFO", "ERROR"]

    if second_arg not in accepted_args:
        raise ValueError(f"Invalid argument: {second_arg}. Accepted values are 'INFO' or 'ERROR'.")

    if second_arg == "INFO":
        for each_line in data:
            if "INFO" in each_line.strip():
                print(each_line.strip())
    elif second_arg == "ERROR":
        for each_line in data:
            if "ERROR" in each_line.strip():
                print(each_line.strip())

except ValueError as ve:
    print(ve)
    sys.exit(1)

except FileNotFoundError as fnfe:
    print(fnfe)
    sys.exit(1)

except Exception as e:
    print(f"An unexpected error occurred: {e}")
    sys.exit(1)

```

Wrong argument count or invalid filter.

Invalid file path.

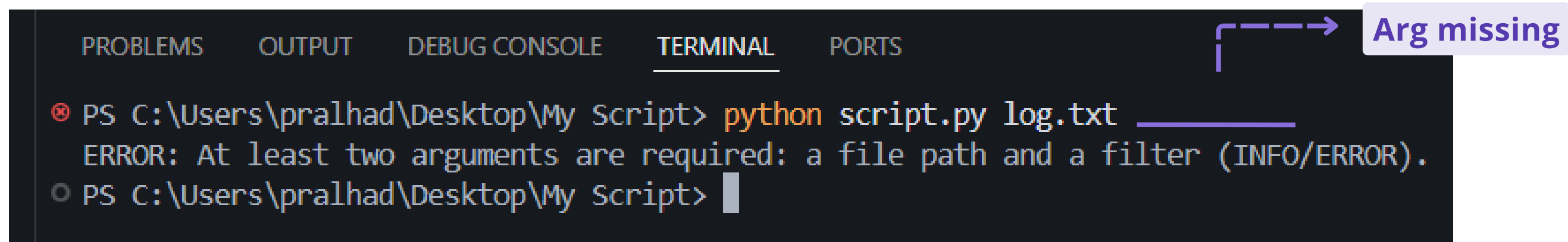
Any other unexpected error.

We'll use **try-except** blocks along with **sys.exit()**.

- Wrap the main logic inside a try block.
- Catch known errors

Testing the Fix

Test Case 1: Too Few Arguments



The screenshot shows a terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The TERMINAL tab is active. The command prompt shows the user running `python script.py log.txt`. The output is an error message: "ERROR: At least two arguments are required: a file path and a filter (INFO/ERROR)." A purple dashed arrow points from a callout box labeled "Arg missing" to the end of the command line.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
⊗ PS C:\Users\pralhad\Desktop\My Script> python script.py log.txt
ERROR: At least two arguments are required: a file path and a filter (INFO/ERROR).
○ PS C:\Users\pralhad\Desktop\My Script>
```

Test Case 2: Invalid File Path



The screenshot shows a terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The TERMINAL tab is active. The command prompt shows the user running `python script.py non_existent_file.txt INFO`. The output is an error message: "ERROR: The file 'non_existent_file.txt' does not exist. Please provide a valid file path." A purple dashed arrow points from a callout box labeled "incorrect file path" to the file path in the command line.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
⊗ PS C:\Users\pralhad\Desktop\My Script> python script.py non_existent_file.txt INFO
ERROR: The file 'non_existent_file.txt' does not exist. Please provide a valid file path.
○ PS C:\Users\pralhad\Desktop\My Script>
```

Test Case 3: Invalid Filter Argument

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
⊗ PS C:\Users\pralhad\Desktop\My Script> python script.py log.txt WARNING  
Invalid argument: WARNING. Accepted values are 'INFO' or 'ERROR'.  
○ PS C:\Users\pralhad\Desktop\My Script> 
```

Invalid

Test Case 4: Typo Error

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
⊗ PS C:\Users\pralhad\Desktop\My Script> python script.py log.txt EROORS  
Invalid argument: EROORS. Accepted values are 'INFO' or 'ERROR'.  
○ PS C:\Users\pralhad\Desktop\My Script> 
```

It should be ERROR not EROORS

All Edge Cases Handled - All Togherther.



```
import sys
import os

try:
    if len(sys.argv) < 3:
        raise ValueError("ERROR: At least two arguments are required: a file path and a filter (INFO/ERROR).")
    if len(sys.argv) > 3:
        raise ValueError("ERROR: Too many arguments. Only two arguments are allowed.")

    first_arg = sys.argv[1]
    if not os.path.isfile(first_arg):
        raise FileNotFoundError(f"ERROR: The file '{first_arg}' does not exist. Please provide a valid file path.")

    with open(first_arg, 'r') as f:
        data = f.readlines()

    second_arg = sys.argv[2]
    accepted_args = ["INFO", "ERROR"]

    if second_arg not in accepted_args:
        raise ValueError(f"Invalid argument: {second_arg}. Accepted values are 'INFO' or 'ERROR'.")

    if second_arg == "INFO":
        for each_line in data:
            if "INFO" in each_line.strip():
                print(each_line.strip())
    elif second_arg == "ERROR":
        for each_line in data:
            if "ERROR" in each_line.strip():
                print(each_line.strip())

except ValueError as ve:
    print(ve)
    sys.exit(1)

except FileNotFoundError as fnfe:
    print(fnfe)
    sys.exit(1)

except Exception as e:
    print(f"An unexpected error occurred: {e}")
    sys.exit(1)
```

Ensures at least 2 arguments are passed

Check if file exists.

Raises FileNotFoundError if the path is invalid.

Only allows "INFO" or "ERROR".

Prints only lines that match the chosen filter

Wrong argument count or invalid filter.

Invalid file path.

Any other unexpected error.

```
import sys
import os

try:
    if len(sys.argv) < 3:
        raise ValueError("ERROR: At least two arguments are required: a file path and a filter (INFO/ERROR).")
    if len(sys.argv) > 3:
        raise ValueError("ERROR: Too many arguments. Only two arguments are allowed.")

    first_arg = sys.argv[1]
    if not os.path.isfile(first_arg):
        raise FileNotFoundError(f"ERROR: The file '{first_arg}' does not exist. Please provide a valid file path.")

    with open(first_arg, 'r') as f:
        data = f.readlines()

    second_arg = sys.argv[2]
    accepted_args = ["INFO", "ERROR"]

    if second_arg not in accepted_args:
        raise ValueError(f"Invalid argument: {second_arg}. Accepted values are 'INFO' or 'ERROR'.")

    if second_arg == "INFO":
        for each_line in data:
            if "INFO" in each_line.strip():
                print(each_line.strip())
    elif second_arg == "ERROR":
        for each_line in data:
            if "ERROR" in each_line.strip():
                print(each_line.strip())

except ValueError as ve:
    print(ve)
    sys.exit(1)

except FileNotFoundError as fnfe:
    print(fnfe)
    sys.exit(1)

except Exception as e:
    print(f"An unexpected error occurred: {e}")
    sys.exit(1)
```

Edge Case 3:

Ensuring the Correct Number of Arguments

Edge Case 0 & 1:

Check if File Exists / File Must Be Valid

Edge Case 2:

The Second Argument Must Be a Valid Filter

Edge Case 4:

Close Program if Exception Fails

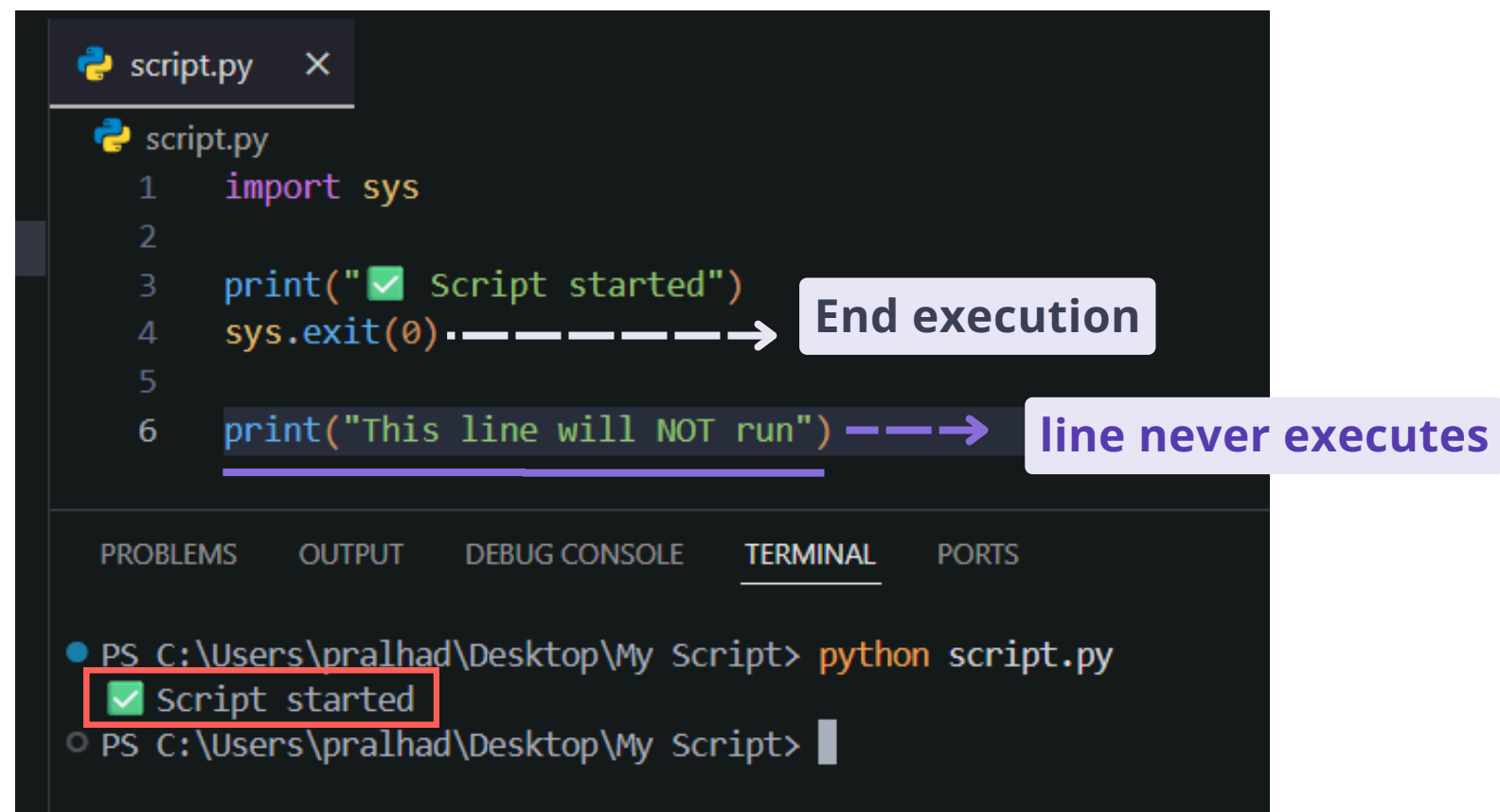
Understanding sys.exit()

- What is sys.exit(0)?
- What is the Number Inside sys.exit()?
- Using sys.exit(0)
- Using sys.exit(1)
- Why We Use sys.exit(0) and sys.exit(1)
- Why This Matters

What is sys.exit(0)?

sys.exit() is a built-in function from the **sys** module that is used to **immediately terminate** a Python program.

- Once it's called, the script **stops running**.
- Any code after **sys.exit()** will be **ignored**.
- It's like saying: *"I'm done. Time to exit!"*



The screenshot shows a code editor with a file named `script.py`. The code contains the following lines:

```
1 import sys
2
3 print("✅ Script started")
4 sys.exit(0)
5
6 print("This line will NOT run")
```

Annotations in the image:

- A dashed arrow points from `sys.exit(0)` to a box labeled "End execution".
- A dashed arrow points from the line `print("This line will NOT run")` to a box labeled "line never executes".

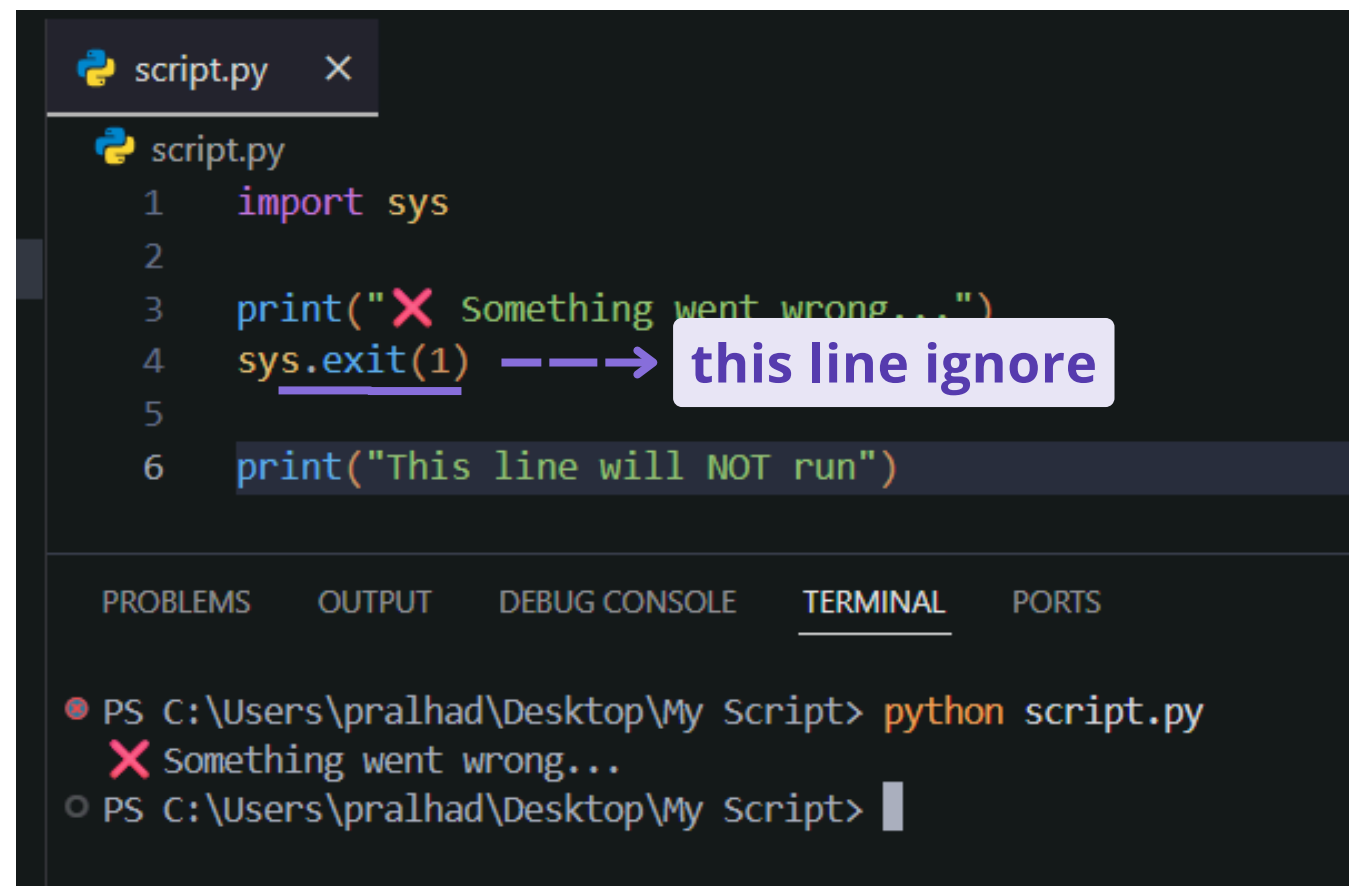
The terminal at the bottom shows the command `python script.py` being executed. The output is:

```
PS C:\Users\pralhad\Desktop\My Script> python script.py
✅ Script started
PS C:\Users\pralhad\Desktop\My Script>
```

The output `✅ Script started` is highlighted with a red box.

What is the Number Inside sys.exit()?

The number you pass into `sys.exit()` is called the **exit code**. It tells the OS or calling program **why** your script exited.



The screenshot shows a code editor with a file named `script.py`. The code contains the following lines:

```
1 import sys
2
3 print("X Something went wrong...")
4 sys.exit(1)
5
6 print("This line will NOT run")
```

A callout box with the text "this line ignore" points to line 4, `sys.exit(1)`. Below the code editor is a terminal window showing the command `python script.py` being executed. The output of the script is `X Something went wrong...`, and the prompt returns to the shell, indicating the script has exited.

And your terminal will report: **Exit code: 1**

Code

Meaning

0

→ ✓ Success – finished normally

1

→ ✗ Error – something went wrong

Using sys.exit(0)

Use sys.exit(0) when:

- The script completes **successfully**
- We want to **exit intentionally** before continuing
- We're signaling to **automated tools** that the run was successful

Example: Exit After Simple User Info

```
import sys

print("This script displays information about a file.")
print("To use it, provide the file path when running the script.")

sys.exit(0)

print("This line will never run.")
```

→ Ends program successfully when no error.

Output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\pralhad\Desktop\test> & C:/Users/pralhad/AppData/Local/P
• This script displays information about a file.
  To use it, provide the file path when running the script.
○ PS C:\Users\pralhad\Desktop\test>
```

Using sys.exit(1)

Use sys.exit(1) when:

- There's a **problem that prevents the script from continuing**
- You want to notify tools or users that an **error occurred**
- You're catching missing input, failed validation, or broken dependencies

```
import sys
import os

if len(sys.argv) < 2:
    print("Error: You must provide a file path.")
    sys.exit(1)

file_path = sys.argv[1]

if not os.path.isfile(file_path):
    print(f"Error: File '{file_path}' not found.")
    sys.exit(1)

print(f"✅ File '{file_path}' found. Continuing...")
```

Output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

⊗ PS C:\Users\pralhad\Desktop\test> python .\script.py
→ Error: You must provide a file path.
⊗ PS C:\Users\pralhad\Desktop\test> python .\script.py hello.txt
→ Error: File 'hello.txt' not found.
○ PS C:\Users\pralhad\Desktop\test> █
```

Exits the program when an error occurs.

File Reader with Handling Edge Cases

- A tool that reads metadata from a .json, .ma, or .nk file before submitting to render.
- If the file doesn't exist or can't be read, the script exits with error, preventing bad jobs from entering the pipeline.

```
import sys
import os

if len(sys.argv) < 2:
    print("Error: Missing argument 'file_path'.")
    print("Usage: python script.py <file_path>")
    sys.exit(1)

file_path = sys.argv[1]

if not os.path.exists(file_path) or not os.path.isfile(file_path):
    print(f"Error: File '{file_path}' does not exist or is not a valid file.")
    sys.exit(1)

try:
    with open(file_path, "r", encoding="utf-8") as file:
        content = file.read()
        print(f"Content of {file_path}:\n")
        print(content)
except Exception as e:
    print(f"Error while reading the file: {e}")
    sys.exit(1)

sys.exit(0)
```

Exits the program when an error occurs.

Ends program successfully when no error.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

⊗ PS C:\Users\pralhad\Desktop\test> python .\script.py hello.txt
Error: File 'hello.txt' does not exist or is not a valid file.
● PS C:\Users\pralhad\Desktop\test> python .\script.py hello.txt
Content of hello.txt:

hello world
○ PS C:\Users\pralhad\Desktop\test> 
```

Why We Use `sys.exit(0)` and `sys.exit(1)`

When we run a Python script or a CLI tool, the script eventually finishes and returns a status code to the operating system (OS).

This status code is important because:

- The **OS or any other program calling script can understand whether it succeeded or failed.**
- Other scripts or automation tools
 - like batch files, shell scripts, render managers, or pipelines
- often **depend on this exit code** to decide what to do next.

Exit Codes

- **sys.exit(0) → Success**

- **0** means “everything worked fine.”
- Convention across all operating systems (**Windows, Linux, Mac**).

```
import sys

print("Script finished successfully.")

sys.exit(0)
# Script finished successfully.
```

exits the program with code 0

- **sys.exit(1) → Error / Failure**

- Non-zero values indicate **something went wrong**.
- **1** is the most common error code, but you can use other numbers (like **2, 3**, etc.) for specific errors if needed.

```
import sys

print("Something went wrong!")
sys.exit(1)
```

exits the program with code 1

Why This Matters

Automation & Pipelines

- In VFX and animation, scripts are often **chained together**
 - e.g., **batch rendering, publishing assets.**
- If one script fails but still returns **0**, the next script might continue blindly, causing **bigger errors** later.

Debugging

- Exit codes **help supervisors**, TDs, or automated systems quickly understand if the script succeeded or failed, without reading the full log.

Industry Standard

- This convention is used in almost all programming languages (**C, C++, Java, Python**).
- It's a universal "**language**" between tools.

Understanding Flags

- What Are Flags?
- Dash (-) vs. Double Dash (--)
- Commonly Used Python Flags
- Good Practices in Flag Design
- CLI Flags

What Are Flags?

A flag is a **special kind of argument** you **pass** to a **program via the command line**. It usually starts with:

- **A single dash (-)** — short form
- **A double dash (--)** — long form

Flags are used to **modify how the program behaves**.

Dash (-) vs. Double Dash (--)

You might've seen both styles in **terminal commands**:



```
python -h  
python --help
```

Short form

long form

Both work — but **why** are there **two styles**?

Single Dash (-)

- Used for **short** flags
- Usually just one letter
- Meant to be **quick** and **easy** to type

Examples:

- **-v** → verbose mode
- **-V** → show Python version
- **-h** → help message

Double Dash (--)

- Used for **long, descriptive** flags
- Easier for beginners to understand
- Makes it clear **what the flag does**

Examples:

- **--version** → shows Python version
- **--help** → displays help instructions

Commonly Used Python Flags

When you run a Python file from the terminal, there are a few flags or options you can use to control how the Python interpreter runs your script.

Common Flags are:

Flag

Description

- | | |
|------------------------|--|
| • -V, --version | → Check the installed Python version |
| • -v, --verbose | → Enable verbose (detailed) output |
| • -h, --help | → Display help and usage options |

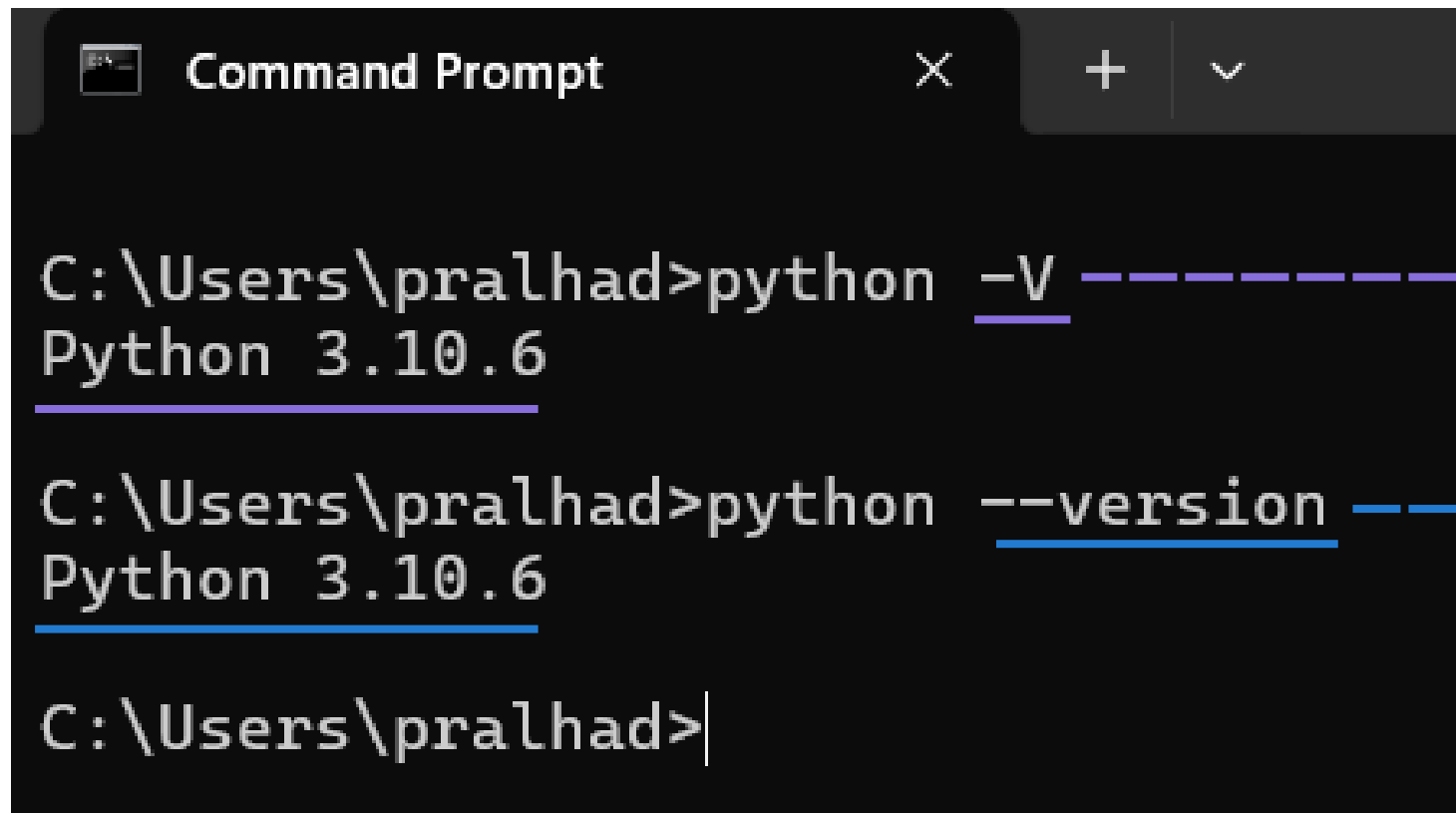
These flags are helpful for:

- Debugging
- Optimization
- Controlling behavior

The -V and --version Flags

Purpose:

Show detailed import traces during startup — useful for debugging module loading.



```
Command Prompt
C:\Users\pralhad>python -V
Python 3.10.6
C:\Users\pralhad>python --version
Python 3.10.6
C:\Users\pralhad>
```

-V is the short form

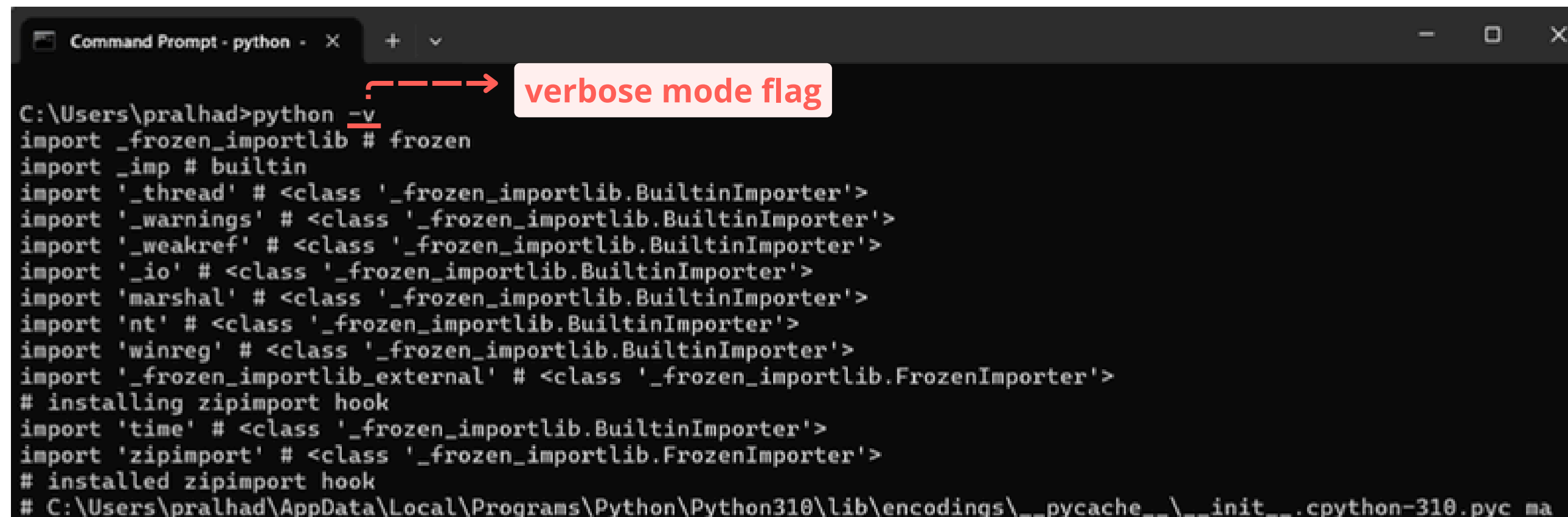
--version is the long form

- Good for **inspecting** startup behavior.

The -v and --verbose Flags

Purpose:

Show detailed import traces during startup — useful for debugging module loading.



```
Command Prompt - python - X + v
C:\Users\pralhad>python -v
import _frozen_importlib # frozen
import _imp # builtin
import '_thread' # <class '_frozen_importlib.BuiltinImporter'>
import '_warnings' # <class '_frozen_importlib.BuiltinImporter'>
import '_weakref' # <class '_frozen_importlib.BuiltinImporter'>
import '_io' # <class '_frozen_importlib.BuiltinImporter'>
import 'marshal' # <class '_frozen_importlib.BuiltinImporter'>
import 'nt' # <class '_frozen_importlib.BuiltinImporter'>
import 'winreg' # <class '_frozen_importlib.BuiltinImporter'>
import '_frozen_importlib_external' # <class '_frozen_importlib.FrozenImporter'>
# installing zipimport hook
import 'time' # <class '_frozen_importlib.BuiltinImporter'>
import 'zipimport' # <class '_frozen_importlib.FrozenImporter'>
# installed zipimport hook
# C:\Users\pralhad\AppData\Local\Programs\Python\Python310\lib\encodings\__pycache__\__init__.cpython-310.pyc ma
```

- **-v** enables **detailed tracing** output, useful for **debugging**
- **--verbose** is **not an official Python flag**, but it is commonly used in many other **command-line tools**
- **Verbosity flags** help you understand **what's happening behind the scenes**

The -h and --help Flags

Purpose:

- universal flag across all tools: **--help**. Python supports both the short form **-h** and the long form **--help**.
- Displays Python usage instructions and available options

```
Command Prompt
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pralhad>python -h
usage: python [option] ... [-c cmd | -m mod | file | -] [arg] ...
Options and arguments (and corresponding environment variables):
-b      : issue warnings about str(bytes_instance), str(bytearray_instance)
         and comparing bytes/bytearray with str. (-bb: issue errors)
-B      : don't write .pyc files on import; also PYTHONDONTWRITEBYTECODE=x
-c cmd  : program passed in as string (terminates option list)
-d      : turn on parser debugging output (for experts only, only works on
         debug builds); also PYTHONDEBUG=x
-E      : ignore PYTHON* environment variables (such as PYTHONPATH)
-h      : print this help message and exit (also -? or --help)
-i      : inspect interactively after running script; forces a prompt even
         if stdin does not appear to be a terminal; also PYTHONINSPECT=x
-I      : isolate Python from the user's environment (implies -E and -s)
-m mod  : run library module as a script (terminates option list)
-O      : remove assert and __debug__-dependent statements; add .opt-1 before
         .pyc extension; also PYTHONOPTIMIZE=x
-OO     : do -O changes and also discard docstrings; add .opt-2 before
         .pyc extension
-q      : don't print version and copyright messages on interactive startup
-s      : don't add user site directory to sys.path; also PYTHONNOUSERSITE
-S      : don't imply 'import site' on initialization
-u      : force the stdout and stderr streams to be unbuffered;
         this option has no effect on stdin; also PYTHONUNBUFFERED=x
-v      : verbose (trace import statements); also PYTHONVERBOSE=x
         can be supplied multiple times to increase verbosity
-V      : print the Python version number and exit (also --version)
```

- **-h** → Short Form
- **--help** → Long Form

Summary

Flag	Function	Notes
• -V, --version	→ Display Python version	→ Both give same result
• -v, --verbose	→ Verbose startup output	→ Built-in debugging info
• -h, --help	→ Show usage and options	→ Good for learning CLI usage

Good Practices in Flag Design

Well-designed tools usually support **both short and long flag formats** to accommodate different users:

-V

Strength → Short, fast

Use When →

- Concise
- Fast to type
- Useful for advanced users

--version  *flag*

Strength → Descriptive, beginner-safe

Use When →

- More descriptive
- Easier to understand at a glance
- Helpful for beginners

Good tools (like **Python**, **Git**, **ffmpeg**) often support **both**:

```
C:\Users\pralhad>python -V
Python 3.10.6

C:\Users\pralhad>python --version
Python 3.10.6

C:\Users\pralhad>
```

-V is the short form

--version is the long form

Result is the same – just **two** ways to access it!

Summary of Conventions

Feature	Single Dash (-)	Double Dash (--)
• Usage	→ Short flags	→ Long, descriptive flags
• Length	→ Typically one letter	→ Multiple letters/words
• Example	→ -v, -V, -h	→ --verbose, --version, --help
• Goal	→ Speed and convenience	→ Clarity and readability
• Best practice	→ Offer both if possible	→ Encourage descriptive options

CLI Flags

- **Flags** are **optional arguments** we pass into command-line tools to **modify how they behave**.
- They act like **settings** or **switches** — toggling features, requesting help, or changing modes.

Common Uses of Flags

Flag	Purpose	Example
• --help, -h	→ Show help or usage instructions	→ <code>python script.py --help</code>
• --version, -V	→ Show program version	→ <code>python --version</code>
• --verbose, -v	→ Enable detailed debug output	→ <code>python script.py --verbose</code>

```
python script.py data.txt --verbose
```

main input

optional flag to print detailed logs

Creating Flags

- What is --help?
- Why It's Important
- Implementing --help
- Using --help
- Running Without --help
- CLI Tool - with Arguments + Flags
- Adding Other Flags
- What Are Positional Arguments/Flags?
- What Are Optional Arguments / Flags?
- Common CLI Flags
- Key Features of Positional Arguments
- Key Features of Optional Flags

What is --help?

Help flag or informational flag

A **standard CLI flag** that shows usage instructions for a command-line tool.

--help is an **informational (optional) flag** — also known as a help flag.

Real-World Usage

Most tools (like **python**, **git**, **ffmpeg**, **aws**) support **--help**.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\pralhad> python --help
usage: C:\Users\pralhad\AppData\Local\Programs\Python\Python310\python.exe [option] ... [-c cmd | -m mod | file | -] [arg
g] ...
Options and arguments (and corresponding environment variables):
-b      : issue warnings about str(bytes_instance), str(bytearray_instance)
         and comparing bytes/bytearray with str. (-bb: issue errors)
-B      : don't write .pyc files on import; also PYTHONDONTWRITEBYTECODE=x
-c cmd  : program passed in as string (terminates option list)
-d      : turn on parser debugging output (for experts only, only works on
         debug builds); also PYTHONDEBUG=x
-E      : ignore PYTHON* environment variables (such as PYTHONPATH)
-h      : print this help message and exit (also -? or --help)
-i      : inspect interactively after running script; forces a prompt even
         if stdin does not appear to be a terminal; also PYTHONINSPECT=x
-I      : isolate Python from the user's environment (implies -E and -s)
-m mod  : run library module as a script (terminates option list)
-O      : remove assert and __debug__-dependent statements; add .opt-1 before
         .pyc extension; also PYTHONOPTIMIZE=x
-OO     : do -O changes and also discard docstrings; add .opt-2 before
         .pyc extension
-q      : don't print version and copyright messages on interactive startup
```

Annotations:

- help flag** points to `--help`
- info about how to use** points to the usage line
- list of all flags we can use with its description** points to the list of options

Purpose:

When a user types **--help**, the script displays:

- What the tool does
- What arguments or options it accepts
- How to run it correctly
- Optional vs required inputs
- Common usage patterns

Why It's Widely Used

- it **allows users to quickly understand how to interact with the tool** without having to dive into the **code**.
- It's a **standard practice** in the **industry** to make scripts **self-explanatory** and **easy to use**.

Implementing --help

Revisiting Our Scenario

We've previously built a simple command-line program that:

It required two arguments:

- **First Argument:** A file name to open (e.g., **log.txt**).
- **Second Argument:** A filter keyword to search for in the file (e.g., **INFO** or **ERROR**).

Now, Let's Add a --help Flag

The goal is to allow users to type:

```
python script.py --help
```

And get a **clear usage guide**, without running the actual logic.

Adding the --help Flag

```
import sys
```

Checks if the user asked for help.

```
if "--help" in sys.argv:
```

```
    print("Usage: script.py <file_path> <filter_keyword>")  
    print("Filter keywords: INFO, ERROR")  
    sys.exit(0)
```

Shows usage and accepted keywords (INFO, ERROR).

```
first_arg = sys.argv[1]
```

1st Args take file input

```
with open(first_arg, 'r') as f:  
    data = f.readlines()
```

```
second_arg = sys.argv[2]
```

2nd Args take keyword input

```
accepted_args = ["INFO", "ERROR"]
```

```
if second_arg in accepted_args:
```

```
    if second_arg == "INFO":
```

```
        for each_line in data:
```

```
            if "INFO" in each_line.strip():
```

```
                print(each_line.strip())
```

```
    elif second_arg == "ERROR":
```

```
        for each_line in data:
```

```
            if "ERROR" in each_line.strip():
```

```
                print(each_line.strip())
```

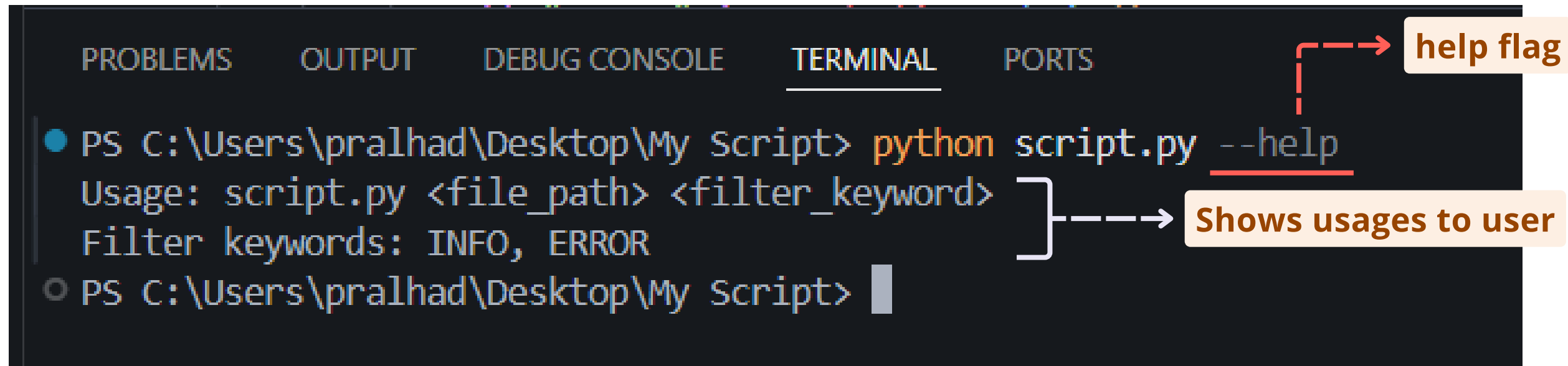
```
else:
```

```
    raise ValueError(f"Invalid argument: {second_arg}. Accepted values are 'INFO' or 'ERROR'.")
```

Raises ValueError if the filter is invalid.

Using --help

Let's test the **--help** flag to see how it works in practice.



The screenshot shows a terminal window with tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The command prompt shows the user running `python script.py --help`. The output displays the usage: `Usage: script.py <file_path> <filter_keyword>` and the filter keywords: `INFO, ERROR`. Annotations include a red arrow pointing to `--help` labeled "help flag" and a bracket pointing to the usage text labeled "Shows usages to user".

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\pralhad\Desktop\My Script> python script.py --help
  Usage: script.py <file_path> <filter_keyword>
  Filter keywords: INFO, ERROR
○ PS C:\Users\pralhad\Desktop\My Script>
```

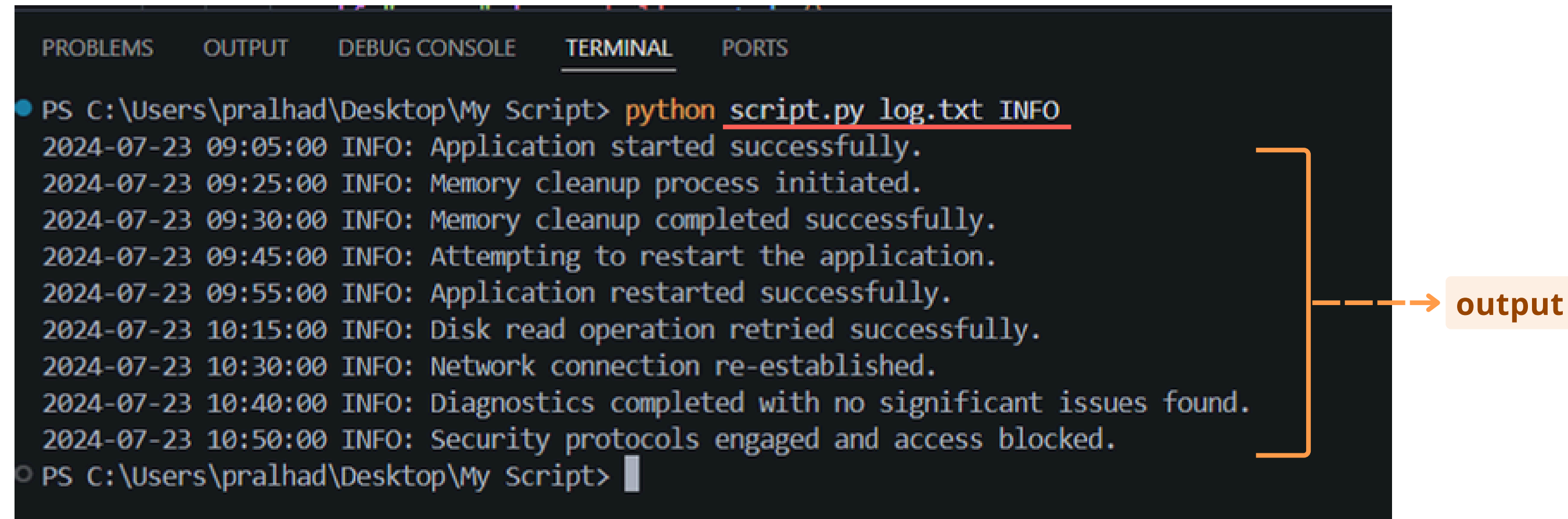
Behavior:

- The program displays the help text.
- It then exits immediately.
- None of the main script logic (like opening files or filtering logs) runs.

This ensures users get clear guidance **without triggering any other part of the program.**

Running Without --help

Now let's run the program as usual — without using **--help**:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\pralhad\Desktop\My Script> python script.py log.txt INFO
2024-07-23 09:05:00 INFO: Application started successfully.
2024-07-23 09:25:00 INFO: Memory cleanup process initiated.
2024-07-23 09:30:00 INFO: Memory cleanup completed successfully.
2024-07-23 09:45:00 INFO: Attempting to restart the application.
2024-07-23 09:55:00 INFO: Application restarted successfully.
2024-07-23 10:15:00 INFO: Disk read operation retried successfully.
2024-07-23 10:30:00 INFO: Network connection re-established.
2024-07-23 10:40:00 INFO: Diagnostics completed with no significant issues found.
2024-07-23 10:50:00 INFO: Security protocols engaged and access blocked.
○ PS C:\Users\pralhad\Desktop\My Script> 
```

✓ Main script logic runs normally when **--help** is not included.

What are we doing here?

We're checking if the user asked for help:

```
if "--help" in sys.argv:
```

If so, we show instructions:

```
print("Usage: script.py <file_path> <filter_keyword>")  
print("Filter keywords: INFO, ERROR")  
sys.exit(0)
```



The **sys.exit(0)** ensures the program **stops immediately**, with exit code **0** meaning “all good.”

Why is this useful?

Aspect

- **Self-explanatory**
- **User-friendly**
- **Efficient**

Spaces

- Helps new users understand the script without reading the source code
- Follows a widely expected standard in CLI tools
- Provides quick feedback without running unnecessary logic

CLI Tool - with Arguments + Flags

So far, our CLI tool includes:

-  **Positional arguments:**
 - **<file_path>** → the log file
 - **<filter_keyword>** → e.g., **INFO, ERROR**
-  **A --help flag** to guide users

```
import sys

if "--help" in sys.argv:
    print("Usage: script.py <file_path> <filter_keyword>")
    print("Filter keywords: INFO, ERROR")
    sys.exit(0)

first_arg = sys.argv[1]
with open(first_arg, 'r') as f:
    data = f.readlines()

second_arg = sys.argv[2]
accepted_args = ["INFO", "ERROR"]

if second_arg in accepted_args:
    if second_arg == "INFO":
        for each_line in data:
            if "INFO" in each_line.strip():
                print(each_line.strip())
    elif second_arg == "ERROR":
        for each_line in data:
            if "ERROR" in each_line.strip():
                print(each_line.strip())
else:
    raise ValueError(f"Invalid argument: {second_arg}. Accepted arguments are: {accepted_args}")
```

Adding Other Flags

Let's Extend Our Script

So far, our CLI tool includes:

-  **Positional arguments:**
 - **<file_path>** → the log file
 - **<filter_keyword>** → e.g., INFO, ERROR
-  **A --help flag** to guide users

Now we'll go a step further by adding **more optional flags**, like:

--option1

--option2

Goal: add these flags

```
import sys

if "--help" in sys.argv:
    print("Usage: script.py <file_path> <filter_keyword>")
    print("Filter keywords: INFO, ERROR")
    sys.exit(0)

first_arg = sys.argv[1]
with open(first_arg, 'r') as f:
    data = f.readlines()

second_arg = sys.argv[2]
accepted_args = ["INFO", "ERROR"]

if second_arg in accepted_args:
    if second_arg == "INFO":
        for each_line in data:
            if "INFO" in each_line.strip():
                print(each_line.strip())
    elif second_arg == "ERROR":
        for each_line in data:
            if "ERROR" in each_line.strip():
                print(each_line.strip())
else:
    raise ValueError(f"Invalid argument: {second_arg}. Accepted
```

These will show **how to detect and react to multiple optional arguments** in a script.

Create Other Flags

```
import sys

if "--help" in sys.argv:
    print("Usage: script.py <file_path> <filter_keyword>")
    print("Filter keywords: INFO, ERROR")
    sys.exit(0)

if '--option1' in sys.argv:
    print("Option 1 selected")

if '--option2' in sys.argv:
    print("Option 2 selected")

try:
    first_arg = sys.argv[1]
    with open(first_arg, 'r') as f:
        data = f.readlines()
except FileNotFoundError:
    print("ERROR: A valid file path is expected as the first argument.")
    sys.exit(1)

second_arg = sys.argv[2]
accepted_args = ["INFO", "ERROR"]

if second_arg in accepted_args:
    if second_arg == "INFO":
        for each_line in data:
            if "INFO" in each_line.strip():
                print(each_line.strip())
    elif second_arg == "ERROR":
        for each_line in data:
            if "ERROR" in each_line.strip():
                print(each_line.strip())
else:
    raise ValueError(f"Invalid argument: {second_arg}. Accepted values are 'INFO' or 'ERROR'.")
```

Prints usage instructions and exits immediately

Prints "Option 1 selected"

Prints "Option 2 selected"

Only accepts INFO or ERROR

Create Other Flags

1. Using --help:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• PS C:\Users\pralhad\Desktop\My Script> python script.py --help
Usage: script.py <file_path> <filter_keyword>
Filter keywords: INFO, ERROR
○ PS C:\Users\pralhad\Desktop\My Script>
```

2. Using --option1:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• PS C:\Users\pralhad\Desktop\My Script> python script.py --option1
Option 1 selected
ERROR: A valid file path is expected as the first argument.
○ PS C:\Users\pralhad\Desktop\My Script>
```

3. Using Multiple Flags:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• PS C:\Users\pralhad\Desktop\My Script> python script.py --option1 --option2
Option 1 selected
Option 2 selected
ERROR: A valid file path is expected as the first argument.
○ PS C:\Users\pralhad\Desktop\My Script>
```

flags

What Are Positional Flags?

- **Positional arguments** are required inputs to a **command-line program**.
- Their meaning is determined solely by position, not by name.

Example Command

```
python myscript.py input.txt output.txt
```

The input file (1st positional argument)

The output file (2nd positional argument)

The script must **know the order**, or it will **misinterpret the values**.

Key Characteristics:

- **Mandatory:** Must be provided
- **Ordered:** Must appear in the correct sequence
- **No prefixes:** Unlike flags, they do not start with **-- or -**

What Are Optional Flags?

- **Optional arguments** (also called **flags** or **options**) are used to modify the **behavior of a script**.
- They are **not required** and can be included in **any order**.

Example Command

```
python myscript.py --help
```

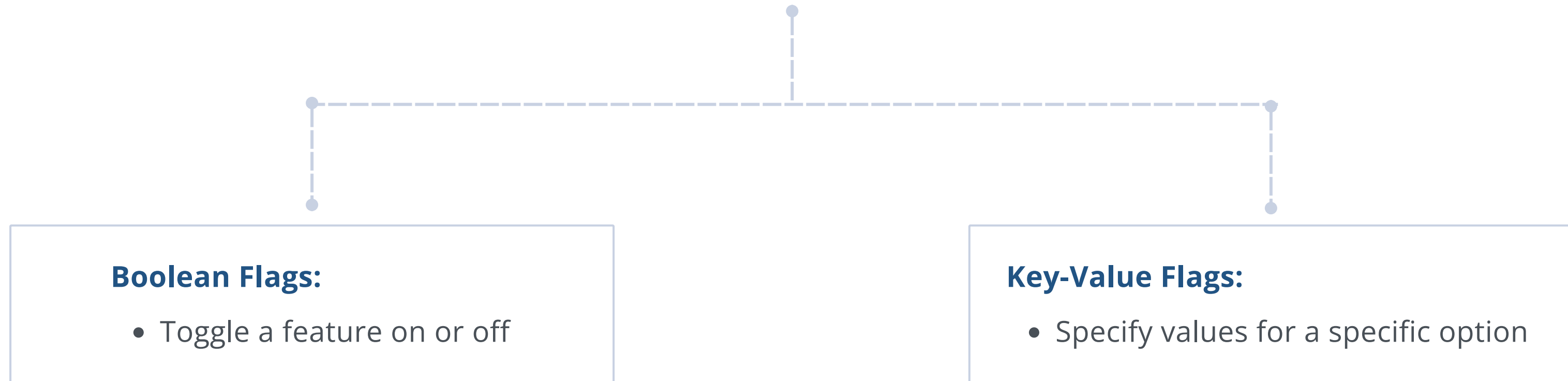


optional flag

Key Characteristics:

- Can be included or skipped
- Use **-** or **--** prefixes
- Usually followed by a value (but not always)
- Make the command **self-explanatory**

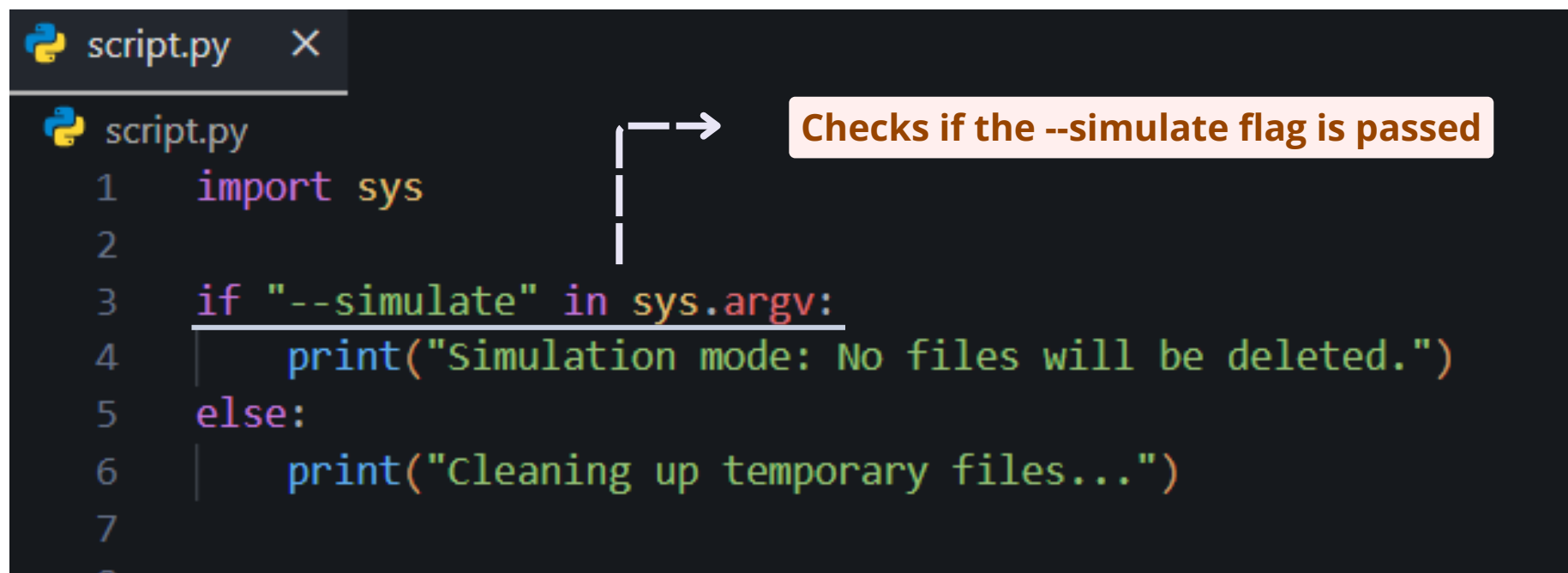
Two Types of Optional Arguments



Boolean Flags

- Used to **turn a feature ON** just by including the flag.
- These do **not** require a value.

Example

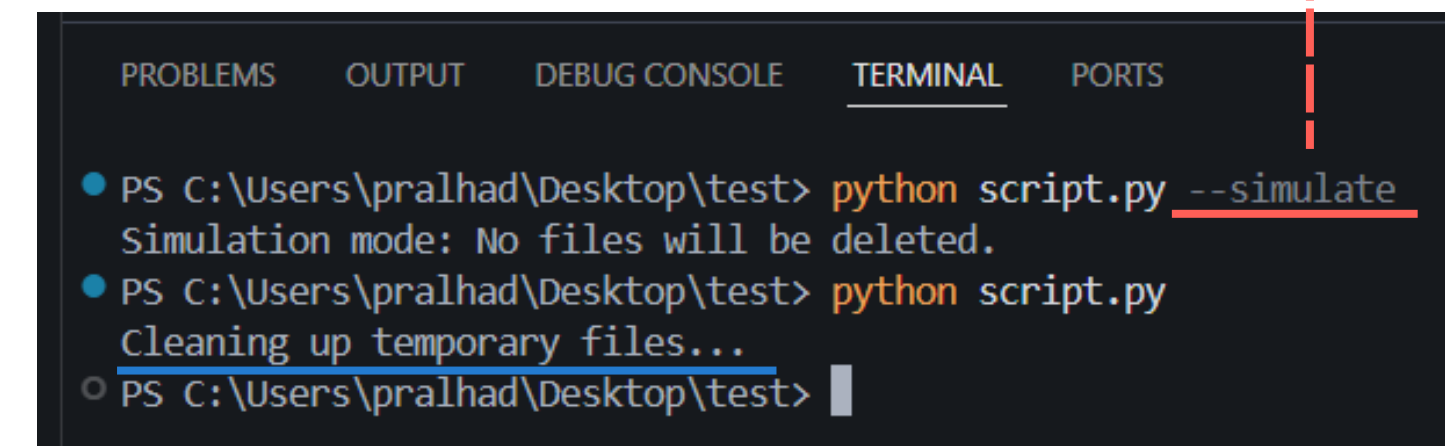


```
script.py X
script.py
1 import sys
2
3 if "--simulate" in sys.argv:
4     print("Simulation mode: No files will be deleted.")
5 else:
6     print("Cleaning up temporary files...")
7
8
```

Checks if the --simulate flag is passed

If you don't include the flag, the script runs normally.

Output



these turn on simulation mode

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
• PS C:\Users\pralhad\Desktop\test> python script.py --simulate
Simulation mode: No files will be deleted.
• PS C:\Users\pralhad\Desktop\test> python script.py
Cleaning up temporary files...
○ PS C:\Users\pralhad\Desktop\test>
```

- **Flag present** → simulation mode
- **Flag absent** → normal execution

Key-Value Flags

Used to **pass a specific value** to the script.

```
import sys

input_path = None
output_path = None

if "--input" in sys.argv:
    idx = sys.argv.index("--input") + 1
    if idx < len(sys.argv):
        input_path = sys.argv[idx]

if "--output" in sys.argv:
    idx = sys.argv.index("--output") + 1
    if idx < len(sys.argv):
        output_path = sys.argv[idx]

if input_path:
    print(f"Input folder: {input_path}")
else:
    print("No input folder provided.")

if output_path:
    print(f"Output folder: {output_path}")
else:
    print("No output folder provided.")
```

Check if --input flag is provided.

Get the value that comes right after --input.

Store that value as the input folder path.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\pralhad\Desktop\test> python script.py --input folderA --output folderB
Input folder: folderA
Output folder: folderB
○ PS C:\Users\pralhad\Desktop\test> 
```

Important Notes

- Using **sys.argv** is fine for very simple scripts, like:

```
python cleanup.py --simulate  
python convert.py --output folderB
```

But it quickly becomes hard to manage when scripts grow.

Example

```
python publish_asset.py --asset chair --version v002 --user alice --notes "Updated UVs" --dry-run
```

- **To handle this manually, you'd have to:**
 - Find and read each flag's value
 - Handle missing or required flags
 - Support multi-word values like "**Updated UVs**"
 - Allow any order
 - Add help messages and defaults

That becomes **messy** and **error-prone** using just **sys.argv**.

What We'll Use Instead

- In future lessons, **we'll switch to:**
 - **argparse** → built-in, structured, and powerful
 - **click** → simpler syntax, good for tools used by artists
- These libraries handle argument parsing **cleanly** and **safely**, which is **essential** in real **VFX pipeline scripts**.

Advantages of Optional Flags

- **Flexibility:** Users can choose which flags to include based on their needs.
- **Customization:** Allows fine-tuning the behavior of a script or command.
- **Clarity:** Named flags like **--verbose** or **--output** make the purpose of each option explicit.

We'll Learn **argparse** & **Click** in Future

Common CLI Flags

GamutX

Flag

Purpose

- | | |
|------------------------|--|
| • --help | → Displays usage instructions or help info |
| • --version | → Shows the version of the program |
| • -v, --verbose | → Enables detailed or debug output |
| • --config | → Specifies a configuration file |
| • --dry-run | → Simulates execution without making changes |

Key Features of Positional Arguments

Mandatory

- Positional arguments are **required** for the program to **run successfully**.
- If they are **missing** (and not given default values), the program will **raise an error**.

Order Matters

- The position of the argument determines what it **represents**.
- **Swapping the order** can lead to **unexpected results or program failure**.

No Prefixes

- These arguments are passed directly, without - or --.
- They are typically file names, configuration paths, or essential input values.

Purpose

- Used to pass **core data** the program needs to operate.
- **Common examples:** input file, output directory, number of threads, etc.

Key Features of Optional Flags

Optional Usage

- Optional flags are **not required** for a script to run (unless the **script enforces them**).
- They allow users to **customize how the script runs**, offering extra functionality or control.

Syntax

- **Short Flags:** Start with a single dash (-) followed by a single letter
 - **Example:** **-v** (for "verbose")
- **Long Flags:** Start with two dashes (--) followed by a full word or phrase
 - **Example:** **--verbose**

Functionality

Optional flags can behave in different ways depending on the script design:

- **Boolean Flags:** Toggle a feature on or off
 - **Example:** **--verbose** (turns on verbose output)
- **Key-Value Flags:** Specify values for a specific option
 - **Example:** **--output result.txt** (sets the output file)

Standard Components of a --help

- Problem Statement
- Sample Help Message
- Help Message Components
- Usage Information
- Options
- Arguments
- Examples
- Why This Help Message Structure Works
- Real World Examples
- Adding a Help Message to the Script
- Why Add --help?

Problem Statement

When we build a **CLI tool**, one of the **most important things** is to include a proper **--help message**.

Problem

```
import sys

if "--help" in sys.argv:
    print("Usage: script.py <file_path> <filter_keyword>")
    print("Filter keywords: INFO, ERROR")
    sys.exit(0)
```

- Only shows **very limited info**, not a full guide.
- Missing clear sections like **Options**, **Arguments**, and **Examples**.
- Doesn't explain what the **tool** actually **does**.
- **Hard to extend** if more flags or features are added.

A well-structured help message is like a **mini user manual** — **clear, short, and useful**.

Sample Help Message

```
import sys

help_msg = """
Usage: script.py [OPTIONS] <file_path> <filter_keyword>

Options:
  --help          Show this help message and exit
  --verbose       Enable detailed output
  --filter=TYPE   Specify a filter type (e.g., INFO, ERROR)

Arguments:
  <file_path>      The path to the input file
  <filter_keyword> Keyword to filter (e.g., INFO, ERROR)

Examples:
  script.py logs.txt INFO
  script.py --verbose logs.txt ERROR
  script.py --help
"""

if "--help" in sys.argv:
    print(help_msg)
    sys.exit(0)

print("Program running...")
```

- A good **--help** message is like **built-in documentation**.
- It guides users by:

Explaining how to use the tool

Listing options and arguments

Showing examples

Help Message Components

Sample Help Message

Usage: script.py [OPTIONS] <file_path> <filter_keyword>

Options:

--help Show this help message and exit
--verbose Enable detailed output
--filter=TYPE Specify a filter type (e.g., INFO, ERROR)

Arguments:

<file_path> The path to the input file
<filter_keyword> Keyword to filter (e.g., INFO, ERROR)

Examples:

script.py logs.txt INFO
script.py --verbose logs.txt ERROR
script.py --help

Usage Section

- Shows how to run the command.

Options Section

- Lists available flags and what they do.

Arguments Sections

- Explains required inputs.

Example Section

- Provides ready-to-use command samples.

Usage Information

Optional Args

Required Args

```
Usage: script.py [OPTIONS] <file_path> <filter_keyword>
```

Options:

```
--help      Show this help message and exit
--verbose   Enable detailed output
--filter=TYPE Specify a filter type (e.g., INFO, ERROR)
```

Arguments:

```
<file_path>   The path to the input file
<filter_keyword> Keyword to filter (e.g., INFO, ERROR)
```

Examples:

```
script.py logs.txt INFO
script.py --verbose logs.txt ERROR
script.py --help
```

What is it?

- Shows the **expected format** of the command.
- **[OPTIONS]** = optional arguments
- **<file_path>** and **<filter_keyword>** = required arguments

Why it Matters:

- Provides a **quick overview** of how to run the script.
- Reduces usage errors from **incorrect order or missing arguments**.

Best Practices:

- Use **[]** for **optional** arguments.
- Use **<>** for **required** arguments.
- Always call it **<file_path>** (not **<file>** or **<filename>**).

Options

GamutX

```
Usage: script.py [OPTIONS] <file_path> <filter_keyword>

Options:
  --help            Show this help message and exit
  --verbose         Enable detailed output
  --filter=TYPE     Specify a filter type (e.g., INFO, ERROR)

Arguments:
  <file_path>       The path to the input file
  <filter_keyword>  Keyword to filter (e.g., INFO, ERROR)

Examples:
  script.py logs.txt INFO
  script.py --verbose logs.txt ERROR
  script.py --help
```

What is it?

- Describes all the **optional flags** our script accepts.
- Each option is paired with a brief explanation of what it does.
- Flags usually start with **--**, and may accept a value
 - (like **--filter=ERROR**).

Why it Matters:

- Helps users **discover features** and customize script behavior.
- Prevents misuse by **explaining what each option does**.

Best Practices:

- Pair each flag with a clear and concise description.
- Show expected values when needed (**--filter=TYPE**).
- (Optional) Support both short **-v** and long **--verbose** forms.

Arguments

GamutX

```
Usage: script.py [OPTIONS] <file_path> <filter_keyword>

Options:
  --help            Show this help message and exit
  --verbose         Enable detailed output
  --filter=TYPE     Specify a filter type (e.g., INFO, ERROR)

Arguments:
  <file_path>       The path to the input file
  <filter_keyword>  Keyword to filter (e.g., INFO, ERROR)

Examples:
  script.py logs.txt INFO
  script.py --verbose logs.txt ERROR
  script.py --help
```

→ What is it?

- Lists **required** inputs the user must provide.
- Explains what each argument means and what values are allowed.

Why it Matters:

- Makes it clear which inputs are required and in what format.
- Reduces errors from missing or incorrect inputs.

Best Practices:

- List arguments in the order they're expected.
- Use examples to clarify expected values
 - e.g., e.g., **INFO, ERROR**

Examples

GamutX

```
Usage: script.py [OPTIONS] <file_path> <filter_keyword>

Options:
  --help            Show this help message and exit
  --verbose         Enable detailed output
  --filter=TYPE     Specify a filter type (e.g., INFO, ERROR)

Arguments:
  <file_path>       The path to the input file
  <filter_keyword>  Keyword to filter (e.g., INFO, ERROR)

Examples:
  script.py logs.txt INFO
  script.py --verbose logs.txt ERROR
  script.py --help
```

What is it?

- Real-world command examples that show **exactly** how to use the script.

Why it Matters:

- Shows how options and arguments work together.
- Gives practical commands users can copy and run.

Best Practices:

- Include common cases,
 - e.g. **script.py logs.txt INFO.**
- Show combos of **flags** and **args**,
 - e.g. **script.py --verbose logs.txt ERROR.**

Summary

```
Usage: script.py [OPTIONS] <file_path> <filter_keyword>
```

Options:

```
--help      Show this help message and exit
--verbose    Enable detailed output
--filter=TYPE Specify a filter type (e.g., INFO, ERROR)
```

Arguments:

```
<file_path>    The path to the input file
<filter_keyword> Keyword to filter (e.g., INFO, ERROR)
```

Examples:

```
script.py logs.txt INFO
script.py --verbose logs.txt ERROR
script.py --help
```

Usage

- **Purpose** - Shows the structure of the command
- **Tips** - Use [] for optional, <> for required

Options

- **Purpose** - Lists available flags and switches
- **Tips** - Be concise and descriptive

Arguments

- **Purpose** - Describes required inputs
- **Tips** - Include examples where helpful

Examples

- **Purpose** - Demonstrates real usage
- **Tips** - Show combinations of options + args

Why This Help Message Structure Works

- A well-designed **--help** message isn't just a convenience
- **it's a crucial user interface** for CLI tools. Here's why this structure is so effective:

◆ Clarity

- **Easy to scan and understand** at a glance.
- Uses **consistent formatting**
 - Use **[]** for **optional** arguments.
 - Use **<>** for **required** arguments.
- Avoids jargon — ideal for both beginners and experienced users.

◆ Consistency

- Follows **standard CLI conventions** users expect:
 - **Usage:** at the top
 - **Options:** for flags
 - **Arguments:** for required inputs
 - **Examples:** at the end
- Makes your script **feel professional** and **user-friendly**.

◆ Comprehensiveness

- It **covers all aspects** a user might need without overwhelming them

Real World Examples

```
$ multipush --help
multipush CLI v1.0.0 (2023-01-01)
```

Shows tool name, version, release date.

Synchronize files between multiple repositories.

DESCRIPTION

Cobra is a CLI library for Go that empowers applications.
This application is a tool to generate the needed files
to quickly create a Cobra application.

Explains what the tool does in plain language.

USAGE

```
$ multipush COMMAND [OPTIONS]
$ multipush SOURCE TARGET [TARGET...] [OPTIONS]
```

Demonstrates syntax for running commands.

EXAMPLE

Shows one or two real-world command usages.

COMMANDS

Use "multipush [COMMAND] --help" for more information about a command.

OPTIONS

Lists available subcommands

Available Commands:

completion	Generate the autocompletion script for the specified shell
config	A brief description of your command
help	Help about any command
serve	A brief description of your command

- *completion, config, serve*

Flags:

-h, --help	help for multipush
-t, --toggle	Help message for toggle

Shows flags and their descriptions -h, -t

Use "multipush [command] --help" for more information about a command.

Adding a Help Message to the Script

"Let's add a detailed help message to our **existing script**. Here's the **updated code**:"

```
import sys

help_text = """
Usage: script.py [OPTIONS] <file_path> <filter_keyword>

Options:
  --help      Show this help message and exit

Arguments:
  <file_path>      The path to the input file
  <filter_keyword> Keyword to filter (e.g., INFO, ERROR)

Examples:
  script.py logs.txt INFO
  script.py --help
"""

if "--help" in sys.argv:
    print(help_text)
    sys.exit(0)

first_arg = sys.argv[1]
with open(first_arg, 'r') as f:
    data = f.readlines()

second_arg = sys.argv[2]
accepted_args = ["INFO", "ERROR"]

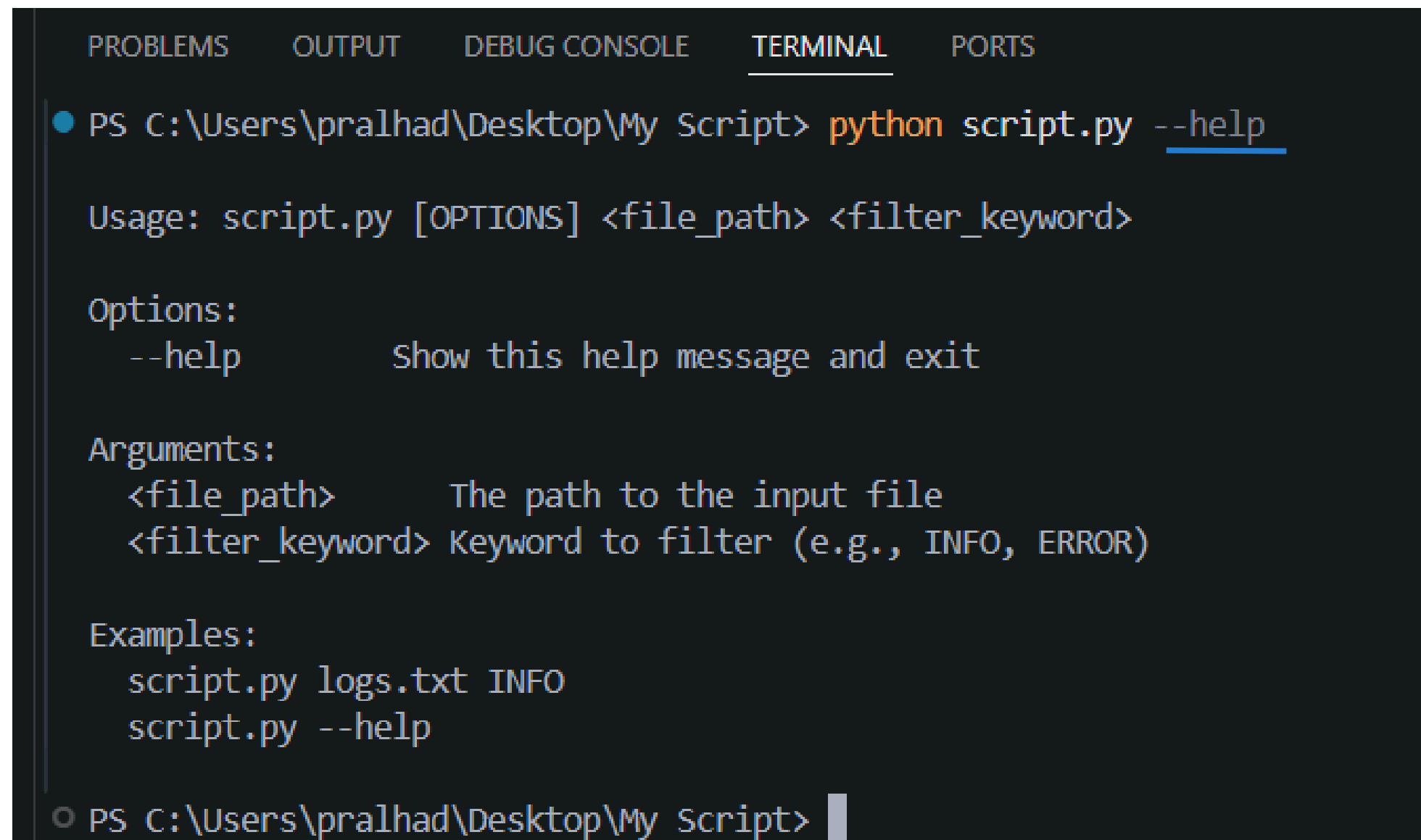
if second_arg in accepted_args:
    if second_arg == "INFO":
        for each_line in data:
            if "INFO" in each_line.strip():
                print(each_line.strip())
    elif second_arg == "ERROR":
        for each_line in data:
            if "ERROR" in each_line.strip():
                print(each_line.strip())
else:
    raise ValueError(f"Invalid argument: {second_arg}. Accepted values are 'INFO' or 'ERROR'.")
```

Holds the detailed help message

Checks if the user asked for help

Demonstrating the Help Message

1. Using the --help Flag



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\pralhad\Desktop\My Script> python script.py --help

Usage: script.py [OPTIONS] <file_path> <filter_keyword>

Options:
  --help          Show this help message and exit

Arguments:
  <file_path>      The path to the input file
  <filter_keyword> Keyword to filter (e.g., INFO, ERROR)

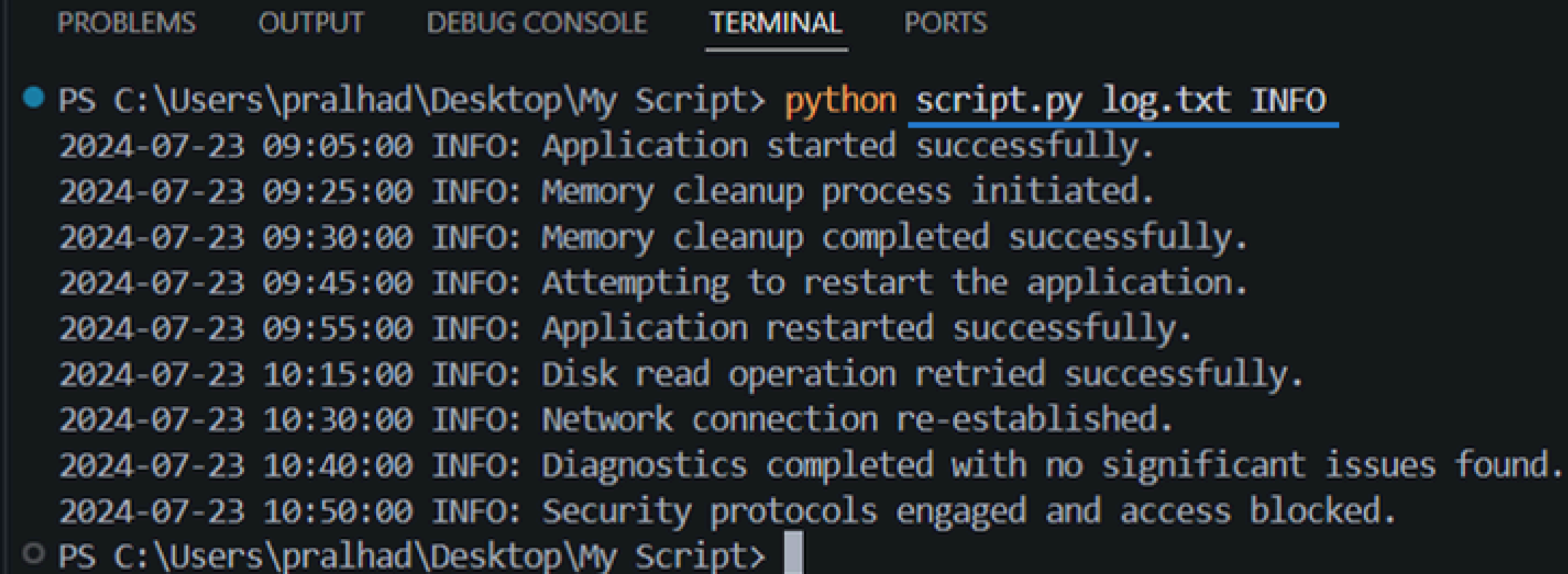
Examples:
  script.py logs.txt INFO
  script.py --help

○ PS C:\Users\pralhad\Desktop\My Script> █
```

✓ The program exits immediately after showing the help message.

2. Running Without the --help Flag

"Let's run the script as intended, without using the **--help** flag."



The screenshot shows a terminal window with a dark background and light-colored text. At the top, there are tabs labeled 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is selected and underlined), and 'PORTS'. The terminal content shows a PowerShell prompt 'PS C:\Users\pralhad\Desktop\My Script>' followed by the command 'python script.py log.txt INFO'. Below this, a series of log messages are displayed, each starting with a timestamp and the word 'INFO:'. The messages describe the application's startup, memory cleanup, restart, disk read operation, network connection, diagnostics, and security protocols. The terminal ends with another PowerShell prompt 'PS C:\Users\pralhad\Desktop\My Script>' and a cursor.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\pralhad\Desktop\My Script> python script.py log.txt INFO
2024-07-23 09:05:00 INFO: Application started successfully.
2024-07-23 09:25:00 INFO: Memory cleanup process initiated.
2024-07-23 09:30:00 INFO: Memory cleanup completed successfully.
2024-07-23 09:45:00 INFO: Attempting to restart the application.
2024-07-23 09:55:00 INFO: Application restarted successfully.
2024-07-23 10:15:00 INFO: Disk read operation retried successfully.
2024-07-23 10:30:00 INFO: Network connection re-established.
2024-07-23 10:40:00 INFO: Diagnostics completed with no significant issues found.
2024-07-23 10:50:00 INFO: Security protocols engaged and access blocked.
○ PS C:\Users\pralhad\Desktop\My Script> █
```

✓ Script behaves as expected.

3. Using --help with Other Arguments

"Let's see what will happen when we pass all arguments **including flag**."

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● PS C:\Users\pralhad\Desktop\My Script> python script.py log.txt INFO --help

Usage: script.py [OPTIONS] <file_path> <filter_keyword>

Options:
  --help          Show this help message and exit

Arguments:
  <file_path>      The path to the input file
  <filter_keyword> Keyword to filter (e.g., INFO, ERROR)

Examples:
  script.py logs.txt INFO
  script.py --help

○ PS C:\Users\pralhad\Desktop\My Script> █
```

✓ The script ignores all other arguments and only displays help.

Why Add --help?

The **--help** flag acts like a **built-in guide** for your script. It makes the tool:

◆ Self-Documenting

- Explains what the script does
- Lists required arguments and optional flags
- Describes input/output expectations

◆ Quick Reference for Everyone

- Saves time for teammates or future-you
- Makes scripts portable and shareable within a team or pipeline

◆ Beginner-Friendly

- Helps new users run the script without reading the code
- Reduces confusion or incorrect usage

◆ A Standard CLI Convention

- Most CLI tools support --help
- Users expect it to work the same way
- It improves the script's professionalism and usability

Summary of Demonstrations

Here's what we've successfully tested so far:

1. Using --help Alone:

```
python script.py --help
```



- ✓ Displays the full help message
- ✓ Exits immediately — no file processing happens

2. Running the Script Normally:

```
python script.py logs.txt INFO
```



- ✓ Reads the file **logs.txt**
- ✓ Filters and displays all lines containing "INFO"

3. Using --help with Other Arguments:

```
python script.py logs.txt INFO --help
```



- ✓ Displays the help message
- ✓ Ignores all other inputs (like **logs.txt** & **INFO**)

Create a Help Flag

Create a simple script with a --help flag.

```
import sys

if "--help" in sys.argv:
    print("""
Usage: python script.py [--help]
Options:
    --help    Show this help message and exit.
Description:
    This script demonstrates how to use the '--help' flag in a Python script.
    Run the script with '--help' to see this message.
Example:
    python script.py --help
""")
    sys.exit(0)
print("Run this script with '--help' to see usage instructions.")
```

Square or Cube Calculator

Use --square or --cube flags to calculate based on user input, including --help.

```
import sys

if "--help" in sys.argv:
    print("""
Usage: python script.py [--square | --cube] <number>
Options:
    --square    Calculate the square of the given number.
    --cube      Calculate the cube of the given number.
    --help      Show this help message and exit.
Description:
    This script takes a number as input and calculates either its square
    or cube based on the flag provided.
Examples:
    python script.py --square 4
    Output: The square of 4 is 16
    python script.py --cube 3
    Output: The cube of 3 is 27
""")
    sys.exit(0)

if len(sys.argv) != 3:
    print("Error: Missing arguments. Use '--help' for instructions.")
    sys.exit(1)

flag = sys.argv[1]

try:
    number = float(sys.argv[2])
except ValueError:
    print("Error: Provide a valid number.")
    sys.exit(1)

if flag == "--square":
    print(f"The square of {number} is {number ** 2}")
elif flag == "--cube":
    print(f"The cube of {number} is {number ** 3}")
else:
    print("Error: Unknown flag. Use '--help' for instructions.")
```

Toggle Debug Mode

```
import sys

if "--help" in sys.argv:
    print("""
Usage: python script.py [--debug]
Options:
  --debug  Enable debug mode.
  --help   Show this help message and exit.
Description:
  This script toggles a debug mode based on the presence of the '--debug' flag.
Examples:
  python script.py --debug
  Output: Debug mode is ON.
  python script.py
  Output: Debug mode is OFF.
""")
    sys.exit(0)

if "--debug" in sys.argv:
    print("Debug mode is ON.")
else:
    print("Debug mode is OFF.")
```

Greet with Uppercase Name

```
import sys

if "--help" in sys.argv:
    print("""
Usage: python script.py [--uppercase] <name>
Options:
  --uppercase  Display the name in uppercase.
  --help       Show this help message and exit.
Description:
  This script greets the user and optionally converts the provided name to uppercase.
Examples:
  python script.py --uppercase Alice
  Output: Hello, ALICE!
  python script.py Alice
  Output: Hello, Alice!
""")
    sys.exit(0)

if len(sys.argv) < 2 or len(sys.argv) > 3:
    print("Error: Invalid arguments. Use '--help' for instructions.")
    sys.exit(1)

if "--uppercase" in sys.argv:
    name = sys.argv[2].upper()
else:
    name = sys.argv[1]
print(f"Hello, {name}!")
```

Number Range Validator

```
import sys

if "--help" in sys.argv:
    print("""
Usage: python script.py <number> --min <min_value> --max <max_value>
Options:
    --min        Specify the minimum valid value.
    --max        Specify the maximum valid value.
    --help       Show this help message and exit.
Description:
    This script validates if a given number is within a specified range.
Examples:
    python script.py 5 --min 1 --max 10
    Output: 5 is within the range [1, 10].
    python script.py 15 --min 1 --max 10
    Output: 15 is outside the range [1, 10].
""")
    sys.exit(0)

if len(sys.argv) != 6 or "--min" not in sys.argv or "--max" not in sys.argv:
    print("Error: Invalid arguments. Use '--help' for instructions.")
    sys.exit(1)

try:
    number = int(sys.argv[1])
    min_value = int(sys.argv[sys.argv.index("--min") + 1])
    max_value = int(sys.argv[sys.argv.index("--max") + 1])
except ValueError:
    print("Error: Invalid numbers provided.")
    sys.exit(1)

if min_value <= number <= max_value:
    print(f"{number} is within the range [{min_value}, {max_value}].")
else:
    print(f"{number} is outside the range [{min_value}, {max_value}].")
```

Batch File Rename Simulator

```
import sys
import os

if "--help" in sys.argv:
    print("""
Usage: python script.py <directory_path> <prefix> [--preview | --rename]
Options:
    --preview    Preview the new file names without renaming.
    --rename     Rename the files with the given prefix.
    --help       Show this help message and exit.
Description:
    This script renames all files in a directory by adding a prefix.
Examples:
    python script.py ./test_folder new_ --preview
    Output: Displays the new file names without renaming.
    python script.py ./test_folder new_ --rename
    Output: Renames the files with the specified prefix.
""")
    sys.exit(0)

if len(sys.argv) != 4 or sys.argv[3] not in ["--preview", "--rename"]:
    print("Error: Invalid arguments. Use '--help' for instructions.")
    sys.exit(1)

directory = sys.argv[1]
prefix = sys.argv[2]
flag = sys.argv[3]

if not os.path.isdir(directory):
    print("Error: Directory not found.")
    sys.exit(1)

files = os.listdir(directory)

for file in files:
    new_name = prefix + file

    if flag == "--preview":
        print(f"Preview: {file} -> {new_name}")
    elif flag == "--rename":
        os.rename(os.path.join(directory, file), os.path.join(directory, new_name))
        print(f"Renamed: {file} -> {new_name}")
```

Palindrome Checker

```
import sys

if "--help" in sys.argv:
    print("""
Usage: python script.py <word> [--ignore-case]
Options:
  --ignore-case  Ignore case when checking for palindrome.
  --help         Show this help message and exit.
Description:
  This script checks if a given word is a palindrome.
Examples:
  python script.py racecar
  Output: 'racecar' is a palindrome.
  python script.py RaceCar --ignore-case
  Output: 'RaceCar' is a palindrome (ignoring case).
""")
    sys.exit(0)

if len(sys.argv) < 2 or len(sys.argv) > 3:
    print("Error: Invalid arguments. Use '--help' for instructions.")
    sys.exit(1)

word = sys.argv[1]

if "--ignore-case" in sys.argv:
    word = word.lower()

if word == word[::-1]:
    print(f"'{sys.argv[1]}' is a palindrome.")
else:
    print(f"'{sys.argv[1]}' is not a palindrome.")
```

Thank You

